



Seleccionando información con Pandas

Selección básica de información

Imagina que estás manipulando información relacionada con la salud de la población, específicamente sobre vacunación, y que recuperaste un archivo que contiene más de 20 mil registros con 15 columnas de información, ¿cómo seleccionarías los datos que hacen referencia a México?, ¿cómo filtrarías las columnas para mantener únicamente la fecha, la vacunación total acumulada, los esquemas completos y parciales, el número de vacunas aplicadas diariamente y la disponibilidad por tipo de vacuna? Esto te conduciría a la aplicación de uno o varios procesos de extracción

Existen diversas formas en las que puedes seleccionar información de un *dataframe* en Pandas. Las más comunes son:

Tipo de acceso	Uso	Resultado
Seleccionar una columna	<code>dataframe[columna]</code>	Series
Seleccionar varias columnas	<code>dataframe [columnas]</code>	DataFrame
Seleccionar filas por condición	<code>dataframe [condición]</code>	DataFrame
Seleccionar filas por segmentación	<code>dataframe [iif:iff]</code>	DataFrame
Seleccionar filas y columnas por condición y etiqueta	<code>dataframe.loc[condicional o etiquetas, etiquetas]</code>	DataFrame
Seleccionar filas y columnas por subíndice	<code>dataframe.iloc[iif:iff, iic: ifc]</code>	DataFrame

Para ejemplificar los diferentes tipos de selección, nuevamente tomaremos como base el archivo de los personajes de la película *Star Wars*, denominado **characters.csv** (descárgalo desde el apartado “Archivos adjuntos” de la plataforma). Este archivo contiene 87 registros y las siguientes columnas: `id`, `name`, `height`, `mass`, `hair_color`, `skin_color`, `eye_color`, `birth_year`, `gender`, `homeworld` y `species`.

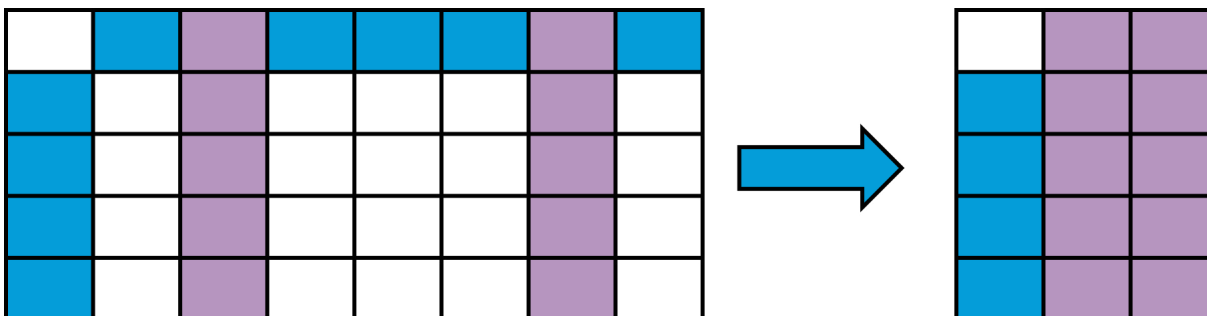
	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
0	1	Luke Skywalker	172	77.0	blond	fair	blue	19BBY	male	Tatooine	Human
1	2	C-3PO	167	75.0	NaN	gold	yellow	112BBY	NaN	Tatooine	Droid
2	3	R2-D2	96	32.0	NaN	white, blue	red	33BBY	NaN	Naboo	Droid
3	4	Darth Vader	202	136.0	none	white	yellow	41.9BBY	male	Tatooine	Human
...	...	...	...	...	...	...	...	...	...	...	...
83	84	Poe Dameron	175	81.0	brown	light	brown	NaN	male	NaN	Human
84	85	BB8	45	60.0	none	none	black	NaN	none	NaN	Droid
85	86	Captain Phasma	170	79.0	NaN	NaN	NaN	NaN	female	NaN	NaN
86	87	Padmé Amidala	165	45.0	brown	light	brown	46BBY	female	Naboo	Human

87 rows x 11 columns

Veamos en qué consiste cada una de las formas de seleccionar información.

Selección de columnas

Este proceso es el más común, ya que la mayoría de las veces es necesario considerar todos los registros existentes, pero no todos sus atributos o características relacionadas.



Para seleccionar una sola columna puedes utilizar corchetes ([]) con el nombre de la columna que deseas recuperar. A continuación, se presenta el *script* para extraer la estatura (*height*) de todos los personajes de *Star Wars*.

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
alturasPersonajesSW = personajesSW["height"]
print('Tamaño del resultado: ', alturasPersonajesSW.shape)
alturasPersonajesSW
```

Tamaño del resultado: (87,)

```
0    172
1    167
2     96
3    202
...
83   175
84    45
85   170
86   165
```

Name: height, Length: 87, dtype: int64

Como puedes observar, se recuperaron 87 registros, pero ahora solamente tenemos la columna con los datos de las alturas. Recuerda que cada columna en un *dataframe* es una serie, por lo que el resultado obtenido queda almacenado en un objeto de este tipo.

El atributo `shape` devuelve el número de filas y columnas de una estructura tabular; pero, aplicado a una serie, por ser unidimensional únicamente devuelve la cantidad de filas. En este ejemplo, `alturasPersonajesSW.shape = (87,)`.

Cuando requieras seleccionar más de una columna del *dataframe*, tienes que modificar la sintaxis de recuperación, pues las columnas deben especificarse en una lista, lo cual se indica entre corchetes. Por ejemplo, si ahora recuperamos del mismo archivo las columnas de altura (`height`) y especie (`species`), entonces tendrías lo siguiente:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
datosPersonajesSW = personajesSW[["height", "species"]]
print('Tamaño del resultado: ', datosPersonajesSW.shape)
datosPersonajesSW
```

Tamaño del resultado: (87, 2)

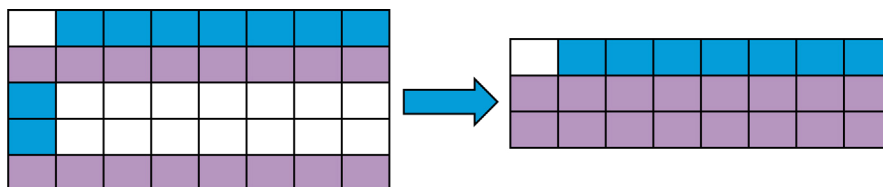
height species		
0	172	Human
1	167	Droid
2	96	Droid
3	202	Human
...	...	...
83	175	Human
84	45	Droid
85	170	NaN
86	165	Human

87 rows x 2 columns

Observa que ahora el resultado es un *dataframe* de dos dimensiones y el atributo `shape` devuelve la cantidad de la fila 87 y de la columna 2 que posee el resultado.

Selección de filas

Veamos ahora el caso en el que deseas mantener todas las características o atributos (columnas) de un registro, pero no todos los registros; es decir, **seleccionar filas por condición**.



Para lograrlo, es necesario que apliques un condicional a alguno de los atributos o características de los registros. El filtrado por la condición especificada incluirá a aquellos registros referidos como filas que la cumplen y excluirá a los que no lo hacen. Debes colocar el condicional dentro de los corchetes y podrás utilizar los operadores relacionales ($<$, $>$, $=$, $!=$, $>=$, $<=$) y los lógicos ($|$, $\&$) que ya conoces. Por ejemplo, podemos obtener todos los personajes de *Star Wars* que sean más bajos de 100 cm, como se muestra en el siguiente *script*:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesBajosSW = personajesSW[personajesSW["height"]<100]
print('Tamaño del resultado: ', personajesBajosSW.shape)
personajesBajosSW
```

Tamaño del resultado: (9, 11)

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
2	3	R2-D2	96	32.0	NaN	white, blue	red	33BBY	NaN	Naboo	Droid
7	8	R5-D4	97	32.0	NaN	white, red	red	NaN	NaN	Tatooine	Droid
18	19	Yoda	66	17.0	white	green	brown	896BBY	male	NaN	Yoda's species
27	28	Arvel Crynyd	90	77.0	brown	fair	brown	NaN	male	NaN	Human
28	29	Wicket Systri Warrick	88	20.0	brown	brown	brown	8BBY	male	Endor	Ewok
44	45	Dud Bolt	94	45.0	none	blue, grey	yellow	NaN	male	Vulper	Vulptereen
71	72	Ratts Tyerell	79	15.0	none	grey, blue	NaN	NaN	male	Aleen Minor	Aleena
72	73	R4-P17	96	63.0	none	silver, red	red, blue	NaN	female	NaN	NaN
84	85	BB8	45	60.0	none	none	black	NaN	none	NaN	Droid

Como puedes observar, se mantienen todas las columnas, pero ahora solamente hay 9 filas en el resultado.

También tienes la posibilidad de utilizar condicionales compuestos como en una instrucción `if`, a través de conectores lógicos como el `or` (`|`) y el `and` (`&`). Es muy importante que cada condicional se coloque entre paréntesis.

El siguiente *script* filtra los registros para que los personajes seleccionados tengan una estatura (`height`) mayor a 170 cm o un peso (`mass`) superior a los 100 kilogramos.

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesAltosPesadosSW =
personajesSW[(personajesSW["height"]>170) |
(personajesSW["mass"]>100)]
print('Tamaño del resultado: ', personajesAltosPesadosSW.
shape)
personajesAltosPesadosSW
```

Tamaño del resultado: (56, 11)

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
0	1	Luke Skywalker	172	77.0	blond	fair	blue	19BBY	male	Tatooine	Human
3	4	Darth Vader	202	136.0	none	white	yellow	41.9BBY	male	Tatooine	Human
5	6	Owen Lars	178	120.0	brown, grey	light	blue	52BBY	male	Tatooine	Human
8	9	Biggs Darklighter	183	84.0	black	light	brown	24BBY	male	Tatooine	Human
...	...	...	...	...	...	...	...	...	...	...	...
79	80	Sly Moore	178	48.0	none	pale	white	NaN	female	Umbara	NaN
80	81	Tion Medon	206	80.0	none	grey	black	NaN	male	Utapau	Pau'an
81	82	Finn	177	87.0	black	dark	dark	NaN	male	NaN	Human
83	84	Poe Dameron	175	81.0	brown	light	brown	NaN	male	NaN	Human

56 rows x 11 columns

En el *dataframe* resultante están presentes todas las columnas, y ahora tienes 56 filas con personajes que tienen una estatura mayor a 170 cm o pesan más de 100 kg.

Para seleccionar filas por segmentación debes colocar, entre corchetes, el segmento en formato [subíndice_inicial:subíndice_final] y podrás recuperar los registros que se encuentren en ese rango. Recuerda que el subíndice final únicamente se emplea como límite, pero el registro en ese subíndice no se incluye en el resultado. Por ejemplo, si seleccionas los subíndices del 10 al 40, obtendrás de resultado un *dataframe* con 30 registros, que van desde el subíndice 10 hasta el 39. Observa la muestra en el siguiente el *script*:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesAltosPesadosSW = personajesSW[10:40]
print('Tamaño del resultado: ', personajesAltosPesadosSW.
shape)
personajesAltosPesadosSW
```

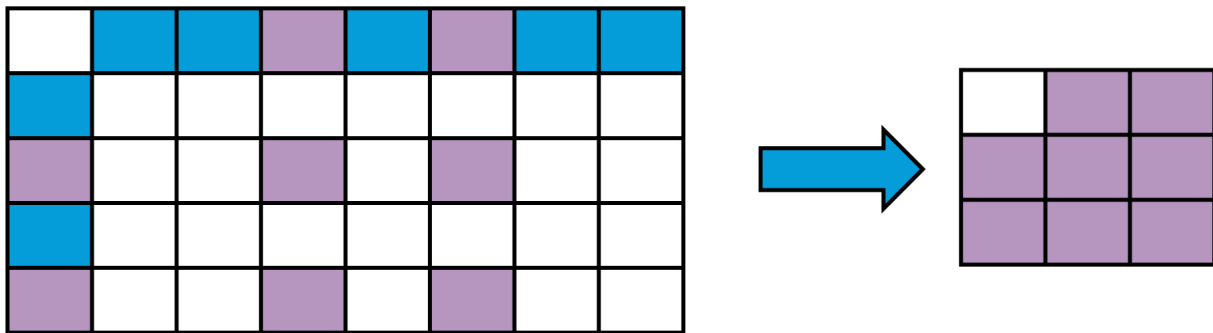
Tamaño del resultado: (30, 11)

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
10	11	Anakin Skywalker	188	84.0	blond	fair	blue	41.9BBY	male	Tatooine	Human
11	12	Wilhuff Tarkin	180	99.0	auburn, grey	fair	blue	64BBY	male	Eriadu	Human
12	13	Chewbacca	228	112.0	brown	NaN	blue	200BBY	male	Kashyyyk	Wookiee
13	14	Han Solo	180	80.0	brown	fair	brown	29BBY	male	Corellia	Human
...	...	...	...	...	...	...	...	...	...	...	...
36	37	Ric Olié	183	102.0	brown	fair	blue	NaN	male	Naboo	NaN
37	38	Watto	137	60.0	black	blue, grey	yellow	NaN	male	Toydaria	Toydarian
38	39	Sebulba	112	40.0	none	grey, red	orange	NaN	male	Malastare	Dug
39	40	Quarsh Panaka	183	88.0	black	dark	brown	62BBY	male	Naboo	NaN

30 rows x 11 columns

Filtrado de filas y columnas

La siguiente selección incluye el filtrado de filas y columnas al mismo tiempo, una combinación de los anteriores. Así, puedes obtener tanto los atributos o características que te interesan, como los registros que deseas.



Para esto, es necesario que utilices los operadores `loc/iloc` antes de los corchetes, los cuales nos permiten que, dentro de los corchetes, especifiquemos las filas que deseamos recuperar; y separado con una coma, las columnas que requerimos.

Utiliza el operador `loc` cuando la selección sea con base en los datos contenidos, es decir, cuando especifiques nombres de columnas, etiquetas de las filas o expresiones condicionales. Por ejemplo, si recuperas únicamente el nombre de los personajes que miden más de 170 cm, tendrías el siguiente *script*:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesAltosSW = personajesSW.
loc[personajesSW["height"]>170, "name"]
print('Tamaño del resultado: ', personajesAltosSW.shape)
personajesAltosSW
```

Tamaño del resultado: (56,)

```
0      Luke Skywalker
3      Darth Vader
5      Owen Lars
8      Biggs Darklighter
...
79     Sly Moore
80     Tion Medon
81     Finn
83     Poe Dameron
```

Name: name, Length: 56, dtype: object

Como ya podrás imaginar, en caso de que requieras recuperar más de una columna, tendrás que emplear una lista para especificarlas. Por ejemplo, si deseas recuperar el nombre y la altura de todos los personajes que miden más de 170 cm, entonces tendrías un *script* como el siguiente:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesAltosSW = personajesSW.
loc[personajesSW["height"]>170, ["name", "height"]]
print('Tamaño del resultado: ', personajesAltosSW.shape)
personajesAltosSW
```

Tamaño del resultado: (56, 2)

	name	height
0	Luke Skywalker	172
3	Darth Vader	202
5	Owen Lars	178
8	Biggs Darklighter	183
...	...	...
79	Sly Moore	178
80	Tion Medon	206
81	Finn	177
83	Poe Dameron	175

56 rows x 2 columns

Una forma muy interesante y útil es cuando necesitas realizar segmentación, pero utilizando las etiquetas. Para esto puedes emplear el operador `loc`. Su funcionamiento es muy particular: se establece la etiqueta inicial y, a continuación, la etiqueta final, como si se tratara de subíndices enteros. Lo que realizará el operador `loc` es colocarse en la etiqueta inicial y avanzar, registro a registro (si la segmentación es por filas) o columna a columna (si la segmentación es por columnas), hasta llegar a la etiqueta indicada como final. Por ejemplo, si tienes un *dataframe* con películas que están indexadas con las etiquetas de los números ordinales, podrías recuperar las películas desde la tercera hasta la octava, mandando estas dos etiquetas al operador `loc`, como se muestra a continuación:

```
import pandas as pd
dataframe = pd.DataFrame(
{
    "Ordinal": ["Primera", "Segunda", "Tercera", "Cuarta",
"Quinta", "Sexta", "Séptima", "Octava", "Novena"],
    "Película": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless", "Terminator", "El Protegido", "Avengers: End
Game", "Stargate", "The Lord of the Rings"],
    "Año": [1977, 1999, 1985, 2006, 1984, 2000, 2019, 1994,
2001],
    "Género": ["Ciencia Ficción", "Ciencia Ficción",
"Ciencia Ficción", "Peleas", "Ciencia Ficción", "Comics",
"Ciencia Ficción", "Ciencia Ficción", "Novela"]
})
dataframe = dataframe.set_index("Ordinal")
dfTerceraOctava = dataframe.loc["Tercera":"Octava"]
```

Original					Selección				
Película Año Género					Película Año Género				
Ordinal					Ordinal				
Primera	Star Wars	1977	Ciencia Ficción		Tercera	Volver al Futuro	1985	Ciencia Ficción	
Segunda	Matrix	1999	Ciencia Ficción		Cuarta	Fearless	2006	Peleas	
Tercera	Volver al Futuro	1985	Ciencia Ficción		Quinta	Terminator	1984	Ciencia Ficción	
Cuarta	Fearless	2006	Peleas		Sexta	El Protegido	2000	Comics	
...	...	...	...		Séptima	Avengers: End Game	2019	Ciencia Ficción	
Sexta	El Protegido	2000	Comics		Octava	Stargate	1994	Ciencia Ficción	
Séptima	Avengers: End Game	2019	Ciencia Ficción						
Octava	Stargate	1994	Ciencia Ficción						
Novena	The Lord of the Rings	2001	Novela						
9 rows x 3 columns									

En cambio, debes utilizar el operador `iloc` cuando la selección sea por subíndices, es decir, con base en la posición de los datos en el *dataframe*. Por ejemplo, si recuperas los registros con subíndices del 20 al 30 y las columnas del 2 al 6, tendrías:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesAltosSW = personajesSW.iloc[20:30, 2:6]
print('Tamaño del resultado: ', personajesAltosSW.shape)
personajesAltosSW
```

Tamaño del resultado: (10, 4)

	height	mass	hair_color	skin_color
20	183	78.2	black	fair
21	200	140.0	none	metal
22	190	113.0	none	green
23	177	79.0	black	dark
...	...	...	...	...
26	150	87.0	auburn	fair
27	90	77.0	brown	fair
28	88	20.0	brown	brown
29	160	68.0	none	grey

10 rows x 4 columns

La selección de información es fundamental en el procesamiento de datos, en donde el filtrado de filas y columnas es la forma más básica de recuperar datos relevantes, pero, como veremos en el siguiente subtema, hay otras formas de poder hacer el filtrado de la información: a través del uso de funciones especializadas para este trabajo.

Otras formas de seleccionar información

Imagina que tienes un *dataframe* con información sobre los fallecimientos de las personas con diabetes y los padecimientos preexistentes de las personas, lo que puede contribuir a identificar las comorbilidades más peligrosas al tener la enfermedad. Sin embargo, todos estos datos están combinados con algunos no relevantes: lugar de nacimiento, estado civil, nivel educativo, entre otros, por lo que es necesario extraer esta información con métodos de selección diferentes a los estudiados.

Una función que nos puede ayudar, que trabaja sobre las series, es `isin`, a la cual se le pasa una lista de valores con los que se compararán los datos de la serie. Esta función regresa como resultado un vector de valores booleanos:

- `True` indica que el valor de la serie se encuentra en la lista
- `False` indica que no se encuentra en la lista.

Este análisis se repite para cada elemento. Por ejemplo, si tienes en una serie el color de ojos de los personajes de *Star Wars* y aplicas la función `isin`, pasándole como parámetro los valores de `blue` y `yellow`, entonces obtendrás el siguiente vector de valores booleanos:

```
personajesSW['eye_color'].isin(['blue', 'yellow'])
```

Serie original		Después de aplicar <code>isin</code>	
0	blue	0	True
1	yellow	1	True
2	red	2	False
3	yellow	3	True
4	brown	4	False
...	...	...	...
82	hazel	82	False
83	brown	83	False
84	black	84	False
85	NaN	85	False
86	brown	86	False

Lo interesante es que, si tienes la serie en un *dataframe* como el siguiente, entonces puedes utilizar la función `isin` para generar el vector de booleanos y emplear este resultado como indexación del *dataframe* que indicará si el registro debe o no mostrarse en la selección de información. Por ejemplo, en este *dataframe* únicamente seleccionas los personajes que tienen el color de ojos `blue` o

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
0	1	Luke Skywalker	172	77.0	blond	fair	blue	19BBY	male	Tatooine	Human
1	2	C-3PO	167	75.0	NaN	gold	yellow	112BBY	NaN	Tatooine	Droid
2	3	R2-D2	96	32.0	NaN	white, blue	red	33BBY	NaN	Naboo	Droid
3	4	Darth Vader	202	136.0	none	white	yellow	41.9BBY	male	Tatooine	Human
4	5	Leia Organa	150	49.0	brown	light	brown	19BBY	female	Alderaan	Human
...	...	...	...	...	...	...	...	...	...	...	...
82	83	Rey	168	77.0	brown	light	hazel	NaN	female	NaN	Human
83	84	Poe Dameron	175	81.0	brown	light	brown	NaN	male	NaN	Human
84	85	BB8	45	60.0	none	none	black	NaN	none	NaN	Droid
85	86	Captain Phasma	170	79.0	NaN	NaN	NaN	NaN	female	NaN	NaN
86	87	Padmé Amidala	165	45.0	brown	light	brown	46BBY	female	Naboo	Human

Recuerda que, si deseas mantener esta selección, deberás guardarla en otro *dataframe* o en el original, según te convenga.

```
personajesSW[personajesSW['eye_color'].isin(['blue', 'yellow'])]
```

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
0	1	Luke Skywalker	172	77.0	blond	fair	blue	19BBY	male	Tatooine	Human
1	2	C-3PO	167	75.0	NaN	gold	yellow	112BBY	NaN	Tatooine	Droid
3	4	Darth Vader	202	136.0	none	white	yellow	41.9BBY	male	Tatooine	Human
5	6	Owen Lars	178	120.0	brown, grey	light	blue	52BBY	male	Tatooine	Human
6	7	Beru Whitesun lars	165	75.0	brown	light	blue	47BBY	female	Tatooine	Human
10	11	Anakin Skywalker	188	84.0	blond	fair	blue	41.9BBY	male	Tatooine	Human
11	12	Wilhuff Tarkin	180	99.0	auburn, grey	fair	blue	64BBY	male	Eriadu	Human
12	13	Chewbacca	228	112.0	brown	NaN	blue	200BBY	male	Kashyyyk	Wookiee
17	18	Jek Tono Porkins	180	110.0	brown	fair	blue	NaN	male	Bestine IV	Human

Lo anterior es equivalente a especificar:

```
personajesSW[(personajesSW['eye_color']=='blue') | (personajesSW['eye_color']=='yellow')]
```

Visualiza la conveniencia del uso de `isin` cuando tengas muchas opciones que comparar.

El método `isin` también se encuentra disponible para realizar búsquedas sobre las etiquetas del *dataframe*. Para ello, únicamente debes indicar el atributo `index` entre el nombre del *dataframe* y la función `isin`, por ejemplo:

```
personajesSW['eye_color'].index.isin(['blue', 'yellow'])
```

En el caso anterior, obtienes un vector con todos los valores `False`, ya que los valores buscados no forman parte de las etiquetas o índices del *dataframe*. Este *dataframe* es no etiquetado y su índice contiene valores numéricos que indican la posición de cada registro.

El método en que está basado el funcionamiento de `isin` se conoce como **indexado booleano**, el cual permite colocar condiciones en lugar de índices o etiquetas. Como resultado, explicado anteriormente, genera un valor booleano indicando si el registro deberá ser incluido o no en la selección. Esta técnica se utilizó también en el subtema anterior cuando se hizo selección de filas por condición.

Veamos otro ejemplo considerando nuevamente el *dataframe* de los personajes de *Star Wars*. Se pueden seleccionar todos los registros donde los personajes sean más bajos de 170 cm:

```
personajesBajos = personajesSW[personajesSW['height']<170]
```


	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
1	2	C-3PO	167	75.0	NaN	gold	yellow	112BBY	NaN	Tatooine	Droid
2	3	R2-D2	96	32.0	NaN	white, blue	red	33BBY	NaN	Naboo	Droid
4	5	Leia Organa	150	49.0	brown	light	brown	19BBY	female	Alderaan	Human
6	7	Beru Whitesun lars	165	75.0	brown	light	blue	47BBY	female	Tatooine	Human
7	8	R5-D4	97	32.0	NaN	white, red	red	NaN	NaN	Tatooine	Droid
18	19	Yoda	66	17.0	white	green	brown	896BBY	male	NaN	Yoda's species
26	27	Mon Mothma	150	87.0	auburn	fair	blue	48BBY	female	Chandril	Human

En algunas ocasiones, los elementos de un *dataframe* pueden contener **valores nulos**, los cuales podrían afectar los resultados de tus procedimientos. Si es necesario eliminar los registros que contienen valores nulos en alguna de sus columnas, puedes utilizar la función `notna`. Esta función realiza un proceso muy similar a `isin`, pero, en lugar de verificar una condición, determina si el elemento en cuestión es nulo o no.

Si tomas como base el *dataframe* original con los personajes de *Star Wars*, podrás observar que la columna `hair_color` contiene elementos nulos representados como NaN.

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
0	1	Luke Skywalker	172	77.0	blond	fair	blue	19BBY	male	Tatooine	Human
1	2	C-3PO	167	75.0	NaN	gold	yellow	112BBY	NaN	Tatooine	Droid
2	3	R2-D2	96	32.0	NaN	white, blue	red	33BBY	NaN	Naboo	Droid
3	4	Darth Vader	202	136.0	none	white	yellow	41.9BBY	male	Tatooine	Human
...	...	...	...	...	...	...	...	...	...	...	...
83	84	Poe Dameron	175	81.0	brown	light	brown	NaN	male	NaN	Human
84	85	BB8	45	60.0	none	none	black	NaN	none	NaN	Droid
85	86	Captain Phasma	170	79.0	NaN	NaN	NaN	NaN	female	NaN	NaN
86	87	Padmé Amidala	165	45.0	brown	light	brown	46BBY	female	Naboo	Human

87 rows x 11 columns

Puedes utilizar la función `notna` para generar el vector con `True` y `False` que sirva como base para la selección de información en los registros del *dataframe*:

```
personajesSW[personajesSW["hair_color"].notna()]
```

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
0	1	Luke Skywalker	172	77.0	blond	fair	blue	19BBY	male	Tatooine	Human
3	4	Darth Vader	202	136.0	none	white	yellow	41.9BBY	male	Tatooine	Human
4	5	Leia Organa	150	49.0	brown	light	brown	19BBY	female	Alderaan	Human
5	6	Owen Lars	178	120.0	brown, grey	light	blue	52BBY	male	Tatooine	Human
...	...	...	...	...	...	...	...	...	...	...	...
82	83	Rey	168	77.0	brown	light	hazel	NaN	female	NaN	Human
83	84	Poe Dameron	175	81.0	brown	light	brown	NaN	male	NaN	Human
84	85	BB8	45	60.0	none	none	black	NaN	none	NaN	Droid
86	87	Padmé Amidala	165	45.0	brown	light	brown	46BBY	female	Naboo	Human

81 rows x 11 columns

Si observas la dimensión del *dataframe* resultante y la comparas con la del *dataframe* original, identificarás que se eliminaron 6 registros. Los registros del *dataframe* que contenían el valor de NaN en la columna `hair_color`, han sido descartados. Si quieres mantener este resultado, recuerda que debes almacenarlo en una variable o sobrescribir el *dataframe* original.

Otra selección que puedes emplear en Pandas es la basada en el tipo de dato.

Se realiza con ayuda de la función `select_dtypes` y trabaja sobre los tipos de datos de las columnas. Esta función recibe como parámetros una lista de tipos de datos que se interpretan de la siguiente manera:

- `include`: Esta lista se utiliza para obtener todas las columnas con los tipos de datos especificados.
- `exclude`: Esta lista incluye los tipos de datos que se deben excluir o ignorar en las columnas.

Para poder ejemplificar esta función con el *dataframe* de personajes de *Star Wars*, primero verifica cuáles son los tipos de datos de las columnas con el atributo `dtypes`.

```
personajesSW.dtypes
```

```
id          int64
name        object
height      int64
mass        float64
hair_color  object
skin_color  object
eye_color   object
birth_year  object
gender      object
homeworld   object
species     object
dtype:      object
```

Si lo que necesitas es recuperar la información numérica podrías emplear `include` con los tipos de datos `int64` y `float64`, lo que sería equivalente a excluir el tipo de dato `object`. Las siguientes dos líneas de código obtienen lo descrito:

```
Opción 1: personajesSW.select_dtypes(include=['int64',
'float64'])
```

```
Opción 2: personajesSW.select_dtypes(exclude=['object'])
```

	id	height	mass
0	1	172	77.0
1	2	167	75.0
2	3	96	32.0
3	4	202	136.0
4	5	150	49.0
...	...	...	...

Además de los tipos de datos empleados por Pandas, también puedes utilizar los tipos de datos genéricos de NumPy, como `number`. Para ver un listado de estos tipos, puedes ejecutar el siguiente fragmento de código que es parte de la documentación oficial de Pandas:

```
import numpy as np
def subdtypes(dtype):
    subs = dtype.__subclasses__()
    if not subs:
        return dtype
    return [dtype, [subdtypes(dt) for dt in subs]]
subdtypes(np.generic)
```

Cuando empleas alguno de los métodos de selección de información, es común que obtengas un subconjunto de los datos; sin embargo, habrá ocasiones en donde necesites que ese resultado tenga el **mismo tamaño** de la serie o *dataframe* original. Para esto existe en Pandas el método `where`.

A continuación, observa una serie con los números del 1 al 10. Utilizando una indexación condicionada, obtendrás un subconjunto de los datos (de tamaño 5), con los números del 5 al 10. Si en su lugar utilizas la función `where`, obtendrás una serie del mismo tamaño que la original, con los valores requeridos mayores a 5, pero con nulo (`NaN`) en aquellos valores que no cumplan la condición.

```
serie
0    1
1    2
2    3
3    4
4    5
5    6
6    7
7    8
8    9
9   10
dtype: int64
```

```
serie[serie>5]
```

```
5      6
6      7
7      8
8      9
9     10
dtype: int64
```

```
serie.where(serie>5)
```

```
0      NaN
1      NaN
2      NaN
3      NaN
4      NaN
5      6.0
6      7.0
7      8.0
8      9.0
9     10.0
dtype: float64
```

La función `where` puede recibir un segundo parámetro que se emplea como valor por defecto en los elementos que no cumplan con la condición.

A continuación, se muestra un *dataframe*, denominado `enterosAleatorios`, de 5 filas por 6 columnas con valores enteros generados aleatoriamente, al cual le aplicarás la función `where` para seleccionar solamente los valores mayores a 50; a los que no cumplan la condición, les asignarás 0.

`enterosAleatorios`

	dat1	dat2	dat3	dat4	dat5	dat6
0	37	52	53	45	51	68
1	75	61	23	94	74	89
2	36	7	31	18	53	75
3	8	70	14	76	74	76
4	26	0	72	69	59	89
5	97	2	30	1	69	13

`enterosAleatorios.where(enterosAleatorios>50, 0)`

	dat1	dat2	dat3	dat4	dat5	dat6
0	0	52	53	0	51	68
1	75	61	0	94	74	89
2	0	0	0	0	53	75
3	0	70	0	76	74	76
4	0	0	72	69	59	89
5	97	0	0	0	69	0

Otra función que resulta muy útil para seleccionar información es la función **query**. Por medio de esta podrás simplificar algunas consultas que pueden resultar muy complejas de manera tradicional (con Python puro y la indexación básica de Pandas). En otras palabras, permite realizar la selección de información en un *dataframe* por medio de una expresión.

Observa el siguiente ejemplo del funcionamiento con un *dataframe* de datos enteros aleatorios, de 10 filas por 3 columnas, en donde se aplica la selección por medio de una condición compuesta para elegir solamente las filas en donde los valores de la columna uno, sean menores a los de la columna dos y tres.

	uno	dos	tres
0	75	96	70
1	65	34	87
2	96	55	13
3	26	63	89
4	67	69	87
5	42	41	68
6	62	77	76
7	86	19	34
8	32	88	25
9	25	1	19

Una forma de realizar la consulta de manera tradicional sería:

```
entAleatorios[(entAleatorios['uno']<entAleatorios['dos']) &
(entAleatorios['uno']<entAleatorios['tres'])]
```

Como puedes observar, es larga y compleja, ya que hay que repetir el nombre del *dataframe* cada vez que se especifica una columna. Resultaría más apropiado, fácil y rápido de codificar si utilizas la función **query** pasando como parámetro el condicional y utilizando los nombres de las columnas de la siguiente manera:

```
entAleatorios.query('uno<dos & uno<tres')
```

	uno	dos	tres
3	26	63	89
4	67	69	87
6	62	77	76

Otra opción que te resultará muy ventajosa cuando estás realizando pruebas de tus algoritmos y de los de terceros, es seleccionar información de un *dataframe* de **manera aleatoria**. Esto lo puedes lograr usando la función `sample`, que tiene los siguientes comportamientos:

- Si no le pasas parámetros, regresará una fila.
- Si le envías el atributo `n` igualado a una cantidad, ese será el número de filas al azar que obtendrás como resultado.
- Si le mandas el atributo `frac`, indicas el porcentaje de datos que deseas recuperar al azar del total del *dataframe*, donde 1 representa todos, 0.75 significa tres cuartas partes de los datos, 0.5 es la mitad, y así sucesivamente.

Por ejemplo, si empleas el *dataframe* de personajes de *Star Wars* y deseas obtener 5 registros al azar, entonces utilizas la función `sample` del *dataframe* con `n = 5`.

	id	name	height	mass	hair_color
0	1	Luke Skywalker	172	77.0	blond
1	2	C-3PO	167	75.0	NaN
2	3	R2-D2	96	32.0	NaN
3	4	Darth Vader	202	136.0	none
4	5	Leia Organa	150	49.0	brown
...	...	...	...	...	...
82	83	Rey	168	77.0	brown
83	84	Poe Dameron	175	81.0	brown
84	85	BB8	45	60.0	none
85	86	Captain Phasma	170	79.0	NaN
86	87	Padmé Amidala	165	45.0	brown

87 rows x 11 columns

```
personajesSW.sample(n=5)
```

	id	name	height	mass	hair_color
19	20	Palpatine	170	75.0	grey
27	28	Arvel Crynyd	90	77.0	brown
77	78	Tarful	234	136.0	brown
43	44	Ayla Secura	178	55.0	none
63	64	Dooku	193	80.0	white

Retomando el ejemplo del *dataframe* con información recopilada sobre los fallecimientos de las personas con diabetes y que tienen asociados todos los padecimientos preexistentes de las personas, podría verse de la siguiente manera:

	Paciente	LugarNacimiento	Edad	NivelEducativo	EstadoCivil	Alergias	Hipertensión	Diabetes
0	Juan Salgado	México	34	Preparatorio	Casado	1	1	0
1	Roberto Macias	Guanajuato	56	Secundaria	Casado	1	1	1
2	Laura Canavaro	Tamaulipas	76	Profesional	Casada	0	0	1
3	Hayde Cervantes	Guerrero	49	Profesional	Divorciada	1	0	0
4	Luis Vela	México	51	Secundaria	Casado	0	1	1
5	Monica Madrigal	Guadalajara	40	Maestría	Divorciada	1	0	0

Para seleccionar la información relevante de las enfermedades, con objetivo de usarla en análisis posteriores, podrías ubicar únicamente las columnas que tienen valores 1 y 0 (que indican la presencia o ausencia de la enfermedad en este conjunto de datos) de la siguiente manera:

```
padecimientos.select_dtypes(include=['number'])
```

	Edad	Alergias	Hipertensión	Diabetes
0	34	1	1	0
1	56	1	1	1
2	76	0	0	1
3	49	1	0	0
4	51	0	1	1
5	40	1	0	0

¡Excelente! Ahora ya puedes poner en práctica lo visto hasta este momento y seleccionar información de múltiples maneras.

| Antes de terminar...

El gran tamaño que poseen las bases de datos u otros orígenes de información en la actualidad, dificultan su procesamiento. La reducción de la dimensión de los *dataframes*, como consecuencia de seleccionar ciertos registros y columnas, hará que tus procesos sean más rápidos y confiables, por lo que, conocer varias técnicas de selección te permitirá encontrar la más adecuada a tu contexto y objetivo, y te facilitará el trabajo en términos de codificación y tiempo.

Reflexiona si en este momento ya eres capaz de:

- Seleccionar los datos relevantes para su manipulación y generar información a partir de los mismos.
- Aplicar selección o filtrado de información por columnas, filas y combinación de ambas.

Recuerda que un conjunto de datos suficiente y descriptivo es el primer peldaño en tus proyectos. La posibilidad de combinar los métodos de selección te abre un sinfín de oportunidades y cubre prácticamente cualquier necesidad de filtrado.

| Material adicional

Pandas

Pandas via. (septiembre de 2024). *Pandas documentation* [Documentación de Pandas]. Pandas. Recuperado el 15 de abril de 2025 de: <https://pandas.pydata.org/docs/index.html>

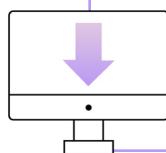
Python

Python Software Foundation. (2025). *Python*. Recuperado el 15 de abril de 2025 de: <https://www.python.org/doc/>

Nota: Los enlaces a las páginas web incluidos en este apartado fueron consultados y verificados por última vez el 15 de abril de 2025 y su disponibilidad depende del autor.

| Glosario

- **Operador:** Símbolo o palabra reservada que indica una operación a realizar entre uno o más operandos. Se utilizan para realizar cálculos, comparar valores, asignar valores, procesar información, etc.
- **Operador lógico:** Operador que se utiliza para combinar dos o más valores booleanos y obtener un resultado. Se utilizan para crear condiciones compuestas o negar una condición.
- **Operador relacional:** Operadores especializados para comparar valores y determinar si una relación es verdadera o falsa.
- **Valores nulos:** Son valores que representan valores de datos ausentes.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.